

Documentação de Integração: MDI-3100-SR com Wemos D1 R32 (ESP32)

1. Arquitetura de Hardware e Riscos Mitigados

A integração do motor de leitura CMOS Opticon MDI-3100-SR com a família ESP32 exige adaptações críticas para contornar gargalos elétricos e lógicos inerentes ao hardware industrial.

- **Pico de Corrente (Brownout):** O scanner pode exigir até 2.5 A no acionamento dos LEDs. Alimentá-lo pelo pino de 3.3V do ESP32 causará reinicializações na placa. A alimentação **deve** ser feita pela linha de 5V.
- **Conexão Física:** O módulo utiliza um conector FPC de 12 pinos (contatos inferiores). O uso de uma placa adaptadora (*breakout board*) FPC para pinos de 2.54mm é obrigatório para o roteamento com o microcontrolador.
- **Controle de Fluxo (Hardware Flow Control):** O firmware do MDI-3100-SR bloqueia a transmissão serial se o pino CTS não estiver em nível lógico baixo. O aterramento forçado deste pino é a solução definitiva.

2. Esquema Elétrico (Pinagem)

A comunicação utiliza a **Serial2** do ESP32, deixando a Serial0 livre para debug no PC.

MDI-3100-SR	Função Original	Conexão na Wemos D1 R32	Justificativa Técnica
1	TRIGn	GPIO 4	Aciona o gatilho de leitura (nível lógico baixo).
2	AIM/WAKEn	GPIO 5	Acorda o módulo do modo <i>Low Power</i> e liga a mira.
7	CTS	GND	Libera o tráfego de dados do scanner (bypassa o controle de fluxo).
8	TxD	GPIO 16 (RX2)	Transmissão do scanner para a recepção do ESP32 (cruzamento).

9	RxD	GPIO 17 (TX2)	Recepção do scanner a partir da transmissão do ESP32.
10	GND	GND	Aterramento comum do circuito.
11	Vcc	5V	Fornecimento de energia primário (suporta picos altos).

Nota: Os pinos não listados (como GR_LEDn, BUZZERn, DWNLDn) possuem resistores de pull-up internos e devem permanecer desconectados (flutuantes).

3. Lógica de Transição de Estados (Energy Management)

O scanner possui uma máquina de estados agressiva para economia de energia. Um pulso simples no pino de gatilho (TRIGn) é ignorado se o dispositivo estiver dormindo. A sequência obrigatória de acionamento via código é:

1. **Acordar (Wake):** Baixar a tensão do pino AIM/WAKEn (GPIO 5) para LOW. Isso acende o LED verde de mira e tira o scanner do modo *Low Power*.
2. **Estabilizar:** Aguardar ~50ms.
3. **Gatilho (Trigger):** Enviar um pulso LOW no pino TRIGn (GPIO 4) por ~50ms e retornar para HIGH. O motor entra em estado de *Read* e acende a iluminação vermelha.
4. **Espera Inteligente (Timeout):** Manter o pino de WAKE em LOW até que a leitura ocorra, limitando esse tempo a um teto razoável (ex: 2 segundos) para não travar o loop principal.
5. **Dormir:** Retornar o pino AIM/WAKEn para HIGH, devolvendo o módulo ao modo de economia de energia.

4. Firmware Consolidado (C++)

O código abaixo implementa a máquina de estados e o fatiamento (*parsing*) dos dados lidos. A comunicação serial está configurada para **115200 bps**, que é o padrão operacional de alta performance do módulo.

```
C++
#include <Arduino.h>

// Mapeamento Serial2 (ESP32)
```

```

#define RXD2 16
#define TXD2 17

// Mapeamento de Controle MDI-3100-SR
#define TRIG_PIN 4
#define WAKE_PIN 5

void setup() {
  Serial.begin(115200);
  // Inicialização da porta serial do Scanner
  Serial2.begin(115200, SERIAL_8N1, RXD2, TXD2);

  // Configuração dos pinos (HIGH por padrão devido ao Pull-up do hardware)
  pinMode(TRIG_PIN, OUTPUT);
  digitalWrite(TRIG_PIN, HIGH);

  pinMode(WAKE_PIN, OUTPUT);
  digitalWrite(WAKE_PIN, HIGH);

  Serial.println("=====");
  Serial.println(" Interface Opticon MDI-3100-SR Ativa  ");
  Serial.println(" Digite TRIGGER para iniciar leitura  ");
  Serial.println("=====");
}

void loop() {
  // 1. TRATAMENTO DE RETORNO DO SCANNER
  if (Serial2.available()) {
    String barcodeData = Serial2.readStringUntil('\n');
    barcodeData.trim(); // Limpeza de buffer (carriage returns e espaços)

    if(barcodeData.length() > 0){
      Serial.print("[SCANNER] Dado Bruto: ");
      Serial.println(barcodeData);

      // PARSING (Tratamento para Sistema de Controle de Acesso/Smart Locker)
      // Exemplo de formato bruto: !N000006310417
      // Estrutura: [!N] Simbologia | [000006] Lote/Facility | [31041] Matrícula | [7] Checksum

      if(barcodeData.length() >= 13) {
        // Extrai o miolo útil baseado em índices fixos
        String matricula = barcodeData.substring(8, 13);
        Serial.print("[SISTEMA] Matrícula Validada: ");
        Serial.println(matricula);

        // Lógica de atuação (ex: abrir relé, enviar requisição HTTP, etc.)

      } else {

```

```

        Serial.println("[ERRO] Formato de código de barras incompatível.");
    }
}
}

```

// 2. RECEPÇÃO DE COMANDOS DO HOST (PC)

```

if (Serial.available()) {
    String command = Serial.readStringUntil('\n');
    command.trim();

    if(command == "TRIGGER") {
        Serial.println("[HARDWARE] Sequência de Acionamento Iniciada...");

        // Fase 1: Wake & Aim
        digitalWrite(WAKE_PIN, LOW);
        delay(50);

        // Fase 2: Trigger Pulse
        digitalWrite(TRIG_PIN, LOW);
        delay(50);
        digitalWrite(TRIG_PIN, HIGH);

        // Fase 3: Timeout Inteligente (Escuta ativa não-blocante)
        unsigned long startTimeout = millis();
        bool leu_com_sucesso = false;

        while(millis() - startTimeout < 2000) {
            if (Serial2.available()) {
                String barcodeData = Serial2.readStringUntil('\n');
                barcodeData.trim();
                if(barcodeData.length() > 0){
                    Serial.print("[SCANNER] Dado Bruto: ");
                    Serial.println(barcodeData);

                    if(barcodeData.length() >= 13) {
                        String matricula = barcodeData.substring(8, 13);
                        Serial.print("[SISTEMA] Matrícula Validada: ");
                        Serial.println(matricula);
                    }
                    leu_com_sucesso = true;
                    break; // Sai do laço imediatamente após o sucesso
                }
            }
        }

        if(!leu_com_sucesso){
            Serial.println("[TIMEOUT] Falha ou ausência de leitura ótica.");
        }
    }
}

```

```

// Fase 4: Sleep
digitalWrite(WAKE_PIN, HIGH);
Serial.println("[HARDWARE] Módulo retornado ao Low Power.");

} else if (command.length() > 0) {
    // Permite o envio de strings nativas de configuração diretamente ao scanner
    Serial.println("[TX] Enviando comando bruto: " + command);
    Serial2.print(command);
    Serial2.print("\r");
}
}
}

```

5. Critérios de Escalabilidade

- **Alteração de Crachás:** Caso o sistema precise ser escalado para ler crachás com formatações dinâmicas (matrículas de 5, 6 ou 7 dígitos misturadas), a extração posicional `substring(8, 13)` precisará ser substituída por uma regra de limpeza dinâmica (ex: apagar os primeiros 8 caracteres absolutos e remover o último caractere relativo `length() - 1`).
- **Comandos Específicos:** Qualquer configuração persistente (como alterar volumes de buzzer de fábrica ou temporização interna) exigirá o envio de códigos seriais descritos no "Universal Menu Book" da fabricante, repassados através da interface criada na etapa de recepção de comandos deste código.